

Decision-aligned eviction-value prediction for learning-augmented caching

Soroush Vahidi^a

^a*Ying Wu College of Computing, New Jersey Institute of Technology, Newark, NJ, USA*

Abstract

Caching with predictive information remains challenging because predictions must be translated into eviction decisions, not merely assessed for accuracy. We study online paging from a candidate-level perspective: at each full-cache miss, the policy scores the items currently in cache and evicts the candidate with smallest predicted downstream harm under a finite-horizon replay target built from a fixed continuation rule. We instantiate this framework as `evict_value_v1` in unweighted paging and evaluate it under a matched protocol against seven baselines spanning classical (LRU, SIEVE, FIFO-Reinsertion) and learned (LRB, 3L-Cache, CACHEUS, HALP) cache replacement, using identical traces, cache capacities, request budgets, and miss accounting. Under this matched comparison, `evict_value_v1` does not outperform any of the seven baselines, losing on a clear majority of matched cells against each. A full-pipeline ablation against reuse-distance, next-arrival, and pairwise-preference supervision likewise finds the eviction-loss instantiation worst or tied-worst of the four. A matched common-model control that changes only the training objective, however, does not support blaming the eviction-loss *label* itself: eviction-loss is nominally best by total misses and is not materially worse than the alternatives. Complementary diagnostics show that the learned scorer agrees with the exact target on 97.5% of decisions, while every measured non-LRU exact-oracle decision has multiple optimal actions (mean optimal-set fraction ≈ 0.991). A deterministic exact-oracle replay loses to LRU on 18 of 21 cells, but this deficit is not robust to valid within-minimum tie-breaking: choosing the LRU-most minimizer never loses to LRU and incurs 413 fewer total misses. A con-

Email address: `sv96@njit.edu` (Soroush Vahidi)

trolled continuation intervention shows a real but partial, family-dependent contribution, and a generic distribution-shift intervention does not itself improve online performance. Finally, a controlled wall-clock timing campaign shows a causal HALP reimplementation is roughly two orders of magnitude slower per decision than the lightweight baselines, while `evict_value_v1`'s own separately measured, single-run runtime is roughly four orders of magnitude slower than the lightweight baselines and substantially slower still than HALP, though it was not part of this controlled repeated-measurement campaign. Taken together, the results characterize finite-horizon eviction-value prediction as an empirically tested but currently unsuccessful supervision-target design whose leading supported limitation is action-underdetermined (tie-degenerate) supervision, rather than a deployment-ready replacement policy.

Keywords: learning-augmented caching, online paging, cache replacement, eviction-value prediction, caching baselines

1. Introduction

1.1. Problem Setting and Motivation

Caching is a fundamental systems mechanism for reducing latency, lowering backend load, and improving throughput in data-intensive services. In online caching, requests arrive sequentially, and when a miss occurs under full capacity, the policy must choose an eviction without access to future requests. Even in the classical paging setting, this local choice has important downstream consequences: a poor eviction can create avoidable future misses, whereas a better choice can preserve useful locality and reduce cumulative cost [1, 2].

Recent work on learning-augmented caching has shown that predictive information can improve online performance when it is informative and incorporated carefully into the replacement rule [3, 4, 5]. At the same time, this literature also makes clear that predictive gains are highly regime dependent. Methods that perform well under one workload, cache size, or prediction model may degrade substantially when predictions are noisy, shifted, or weakly aligned with the actual replacement objective [6, 7]. More broadly, this reflects a central theme in algorithms with predictions: predictive information is useful only when it improves the decision that the online algorithm must actually make [8].

This observation is especially important for practical cache control. At a full-cache miss, the relevant question is not merely whether a predictor should be trusted in the abstract, but which currently cached item is safest to remove. The replacement problem is therefore inherently a candidate-selection problem, and from this perspective the learning target matters. If learning is organized only around a coarse proxy, such as a trust signal or another indirect future hint, the resulting policy may remain weakly aligned with the eviction decision itself. By contrast, a candidate-level target asks a more direct question: what downstream harm is associated with evicting each available item under the current decision context?

This paper studies that candidate-centric view, modeling each full-cache miss as a structured comparison among the current eviction candidates rather than as a single global trust decision about a predictive hint. Section 1.2 states the resulting research objective and scope.

1.2. Research Objective and Scope

Building on the candidate-centric premise introduced above, this paper formulates and directly tests a seemingly natural decision-aligned target for cache eviction – the finite-horizon downstream cost of evicting each candidate – and asks whether structural alignment with the eviction decision is, by itself, sufficient for a learned target to be useful, rather than assuming that alignment guarantees good online behavior.

We organize this investigation around three properties a supervision target for online eviction can possess. These are presented as the conceptual lens used in this study, not as formally established universal definitions. *Decision alignment* means the supervision target corresponds directly to the downstream eviction action or cost, rather than to an indirect proxy that must still be translated into a decision. *Decision informativeness* means the target provides enough separation among the currently available candidates to select meaningfully among them, rather than leaving the decision effectively unconstrained. *Deployment efficiency* means the resulting scoring and control procedure has acceptable online computational cost relative to the alternatives it is compared against. In the framework studied here, decision alignment is built into the proposed target by construction; decision informativeness and deployment efficiency are both empirically investigated rather than assumed.

Within this perspective, the paper has three aims, each corresponding to one of the properties above. First, it formulates eviction as a candidate-

level value-prediction problem that is decision-aligned by construction, instantiated as a finite-horizon eviction-value predictor. Second, it develops an empirical framework for testing whether this decision-aligned target is also informative and competitive in practice, comparing it against classical, predictive, robust, and learned reference policies, and against alternative candidate-level objectives, under a common replay protocol. Third, it diagnoses when and why the target is, or is not, informative enough to induce useful online eviction decisions, including the roles of horizon truncation, target degeneracy, continuation mismatch, and deployment cost. The optional fallback mechanism is retained as an unvalidated implementation extension, not as part of these three empirical aims. Taken together, these aims position the work as a methodological and empirical study of whether decision alignment alone is sufficient for effective cache control.

The scope of the paper is intentionally focused on standard online paging with fixed cache capacity and unit miss cost. We do not present a competitive-ratio guarantee for the learned policy, nor claim universal superiority over alternative learning-augmented formulations. The contribution is instead a decision-aligned supervision target, a candidate-scoring framework built on that target, and a direct empirical test of whether decision alignment translates into decision informativeness and deployment efficiency.

1.3. Main Contributions

The main contributions of this paper are as follows.

1. We formulate learning-augmented paging from a candidate-level decision perspective, in which each full-cache miss is treated as a structured comparison among the items currently stored in the cache, and instantiate it as a finite-horizon eviction-value predictor, `evict_value_v1`.
2. We evaluate `evict_value_v1` under a matched evaluation protocol – identical traces (verified by hash), capacities, request windows, and miss accounting – against seven classical and learned cache-replacement baselines, including a direct comparison with LRB and 3L-Cache as requested by reviewers. Under this matched comparison, `evict_value_v1` does not outperform any of the seven baselines, a finding we report candidly rather than qualify away.
3. We directly test the premise that the proposed eviction-loss target is better aligned with the eviction decision than alternative finite-horizon objectives. A full-pipeline ablation against reuse-distance, next-arrival,

and pairwise-preference supervision finds the eviction-loss instantiation of `evict_value_v1` worst or tied-worst of the four. A subsequent matched common-model control, holding architecture, features, folds, and seed fixed and changing only the training objective, does *not* support the further claim that the eviction-loss training objective itself is the cause: eviction-loss is nominally best by aggregate misses and is not materially worse than the alternatives.

4. We provide a mechanistic diagnosis combining an exact-oracle replay, a target-degeneracy audit, a learned-versus-exact agreement analysis, and a tie-aware follow-up that varies how exact minimizers are resolved. This evidence disfavors gross model-fitting failure and insufficient training data as primary explanations. The horizon-4 target is highly action-underdetermined: every measured non-LRU exact-oracle decision has multiple optimal actions. The previously reported deterministic exact-oracle deficit versus LRU is therefore a tie-breaking confound, not proof that the exact target intrinsically performs worse than LRU. We further isolate, via a controlled causal ablation, the specific contribution of a mismatch between the label-construction continuation rule and the policy’s deployed trajectory, finding a real but partial and family-dependent effect, and show through a companion distribution-shift-correction experiment that reducing a generic measure of this mismatch does not by itself improve online performance.
5. We report a controlled, repeated-measurement wall-clock timing characterization showing the learned policy’s per-decision cost is orders of magnitude higher than the lightweight baselines, and combine this evidence with the negative comparison result above to identify concrete lessons for future learning-augmented eviction-target design, rather than presenting a demonstrated deployment-ready replacement policy.

1.4. Related Work

The classical foundation of this problem lies in online paging and cache replacement, where eviction decisions must be made without access to future requests. Belady’s offline rule provides the standard clairvoyant benchmark, while competitive paging established the canonical worst-case framework for reasoning about online replacement quality [1, 2]. These works capture the central difficulty that remains in modern cache control: the quality of an eviction decision depends on future reuse, yet the decision itself must be made online.

Learning-augmented caching extends this setting by allowing the online algorithm to exploit predictive information. In the paging literature, this line has focused primarily on advice such as next-arrival predictions, predicted actions, and limited future-oriented queries, together with robustness guarantees when the advice is inaccurate [3, 9, 10, 5]. Closely related work on untrusted predictions and robust online algorithms studies how predictive decisions can be combined with safer fallback behavior or expert-style switching without sacrificing worst-case protection [4, 6, 7, 11]. Our work is aligned with this literature in its concern for robustness, but differs in emphasis: rather than centering the design on trust in a hint or on policy-level switching alone, we study a candidate-level supervision target tied directly to the eviction decision itself.

A second relevant line comes from learned cache-replacement systems that use future-derived supervision to guide eviction. PARROT learns to imitate Belady-style eviction behavior from trace-derived oracle decisions, while LRB and Raven use Belady-guided future-access prediction to drive learned replacement decisions [12, 13, 14]. Mockingjay moves beyond binary Belady-style labels by learning finer-grained reuse-time estimates for eviction ordering [15]. HALP is especially relevant because it uses candidate-level preference learning from future re-access outcomes rather than direct action imitation [16]. 3L-Cache combines a lightweight learned scorer with bidirectional candidate sampling and adaptive training controls [17], while CACHEUS combines learned expert selection with adaptive weighting across complementary eviction experts [18]. LRB, 3L-Cache, CACHEUS, and HALP are included as direct empirical comparisons in Section 3.4, with implementation-fidelity caveats disclosed there and in Section 4.3. These methods establish that candidate-level or future-derived learning is an existing design space; our narrower distinction is the particular finite-horizon counterfactual downstream-miss target, not candidate-level learning itself.

This distinction matters because, in many prior formulations, the learned quantity remains an intermediate proxy that must still be translated into an eviction action. Such proxies include oracle action labels, reuse-distance estimates, next-arrival forecasts, and pairwise candidate orderings [12, 15, 14, 16]. By contrast, our formulation assigns each available candidate a counterfactual finite-horizon downstream-miss quantity. This is structurally decision-aligned, but structural alignment does not imply decision informativeness or good online performance. The framing is conceptually related to decision-focused learning and predict-then-optimize work [19, 20, 21]; these

foundations motivate the distinction between predictive accuracy and downstream decision quality, but do not establish a cache-specific theory for our target. Predict-then-optimize theory also treats multiple optimal solutions as a form of degeneracy and studies distance-to-degeneracy or margin concepts [22]; we use that terminology only as a conceptual connection, not as a theorem about our discrete replay process.

The paper is also related to recent finite-horizon predictive approaches such as MUSTACHE, which uses multi-step-ahead page prediction to improve eviction decisions [23]. That work is particularly relevant because it introduces an explicit forecasting horizon. Our method differs in what is supervised: rather than predicting future pages or arrival times and then deriving eviction decisions from those forecasts, we train directly on a finite-horizon eviction-cost target defined at the candidate level. MAT makes a complementary efficiency tradeoff by using a heuristic tail filter to restrict when learned predictions are needed; its authors report eight production workloads and a reduction from 63 to 2 predictions per eviction in the abstract, but we use it here only for its qualitative candidate-filtering distinction because it is an arXiv preprint rather than a peer-reviewed cache-systems publication [24].

Recent systems also emphasize learning granularity and execution cost. GL-Cache learns at the group level to amortize inference across objects, while LAH/S4-FIFO learns cache-level parameters for a static heuristic and separates occasional control-plane learning from the high-speed data plane [25, 26, 27]. Merlin provides a further recent example of fine-grained workload characterization combined with explicit separation of component responsibilities [27]. These approaches reinforce a practical systems lesson relevant to our results: learned caching must jointly provide useful decision information, favorable online miss behavior, and acceptable hot-path cost. S3-FIFO and SIEVE provide important non-ML reference points for the same efficiency pressure: both use simple FIFO-style structures to reduce per-hit or per-eviction overhead while retaining competitive cache efficiency [28, 29]. Our empirical comparison includes SIEVE and FIFO-Reinsertion as lightweight baselines.

2. Main Method

2.1. System Model and Preliminaries

We study online caching in the standard unweighted paging setting. Let

$$\sigma = (r_1, r_2, \dots, r_T) \quad (1)$$

denote the request sequence, where each request $r_t \in \mathcal{P}$ refers to an item in a finite universe $\mathcal{P}$. The cache has capacity k , and its content immediately before serving request r_t is denoted by $C_t \subseteq \mathcal{P}$ with $|C_t| \leq k$. If $r_t \in C_t$, the request is a hit; otherwise, it is a miss. When a miss occurs and the cache is full, that is, when $|C_t| = k$, the policy must evict one cached item before inserting r_t [1, 2].

We focus on the unweighted setting because it isolates the eviction-choice problem while preserving clean comparison with classical, predictive, and robust baselines. Each miss incurs unit cost, so performance is measured primarily by total misses. If $m_t \in \{0, 1\}$ denotes the miss indicator at time t , then the total cost over the sequence is

$$\text{Cost}(\sigma) = \sum_{t=1}^T m_t. \quad (2)$$

Learning-augmented caching enriches this online setting with predictive information that may help guide replacement decisions. Prior work has considered advice in the form of predicted next-arrival times, predicted cache states, and limited access to future-oriented signals [3, 4, 5]. In this paper, such information is treated as auxiliary decision-time context rather than as an instruction that should be followed directly. The goal is therefore not only to assess whether a predictor is trustworthy, but to determine how available predictive context should influence the eviction decision at a full-cache miss.

Formally, when a miss occurs at time t and $|C_t| = k$, the eviction candidates are the items currently stored in the cache:

$$\mathcal{V}_t = C_t. \quad (3)$$

The online action is to select some $q \in \mathcal{V}_t$ for eviction. Classical policies define this action through fixed rules such as recency or marking, whereas predictive policies often derive it from a forecast or advice signal [2, 3]. Our

perspective is different: we evaluate an eviction by its downstream effect on future misses.

To make this idea operational, we define a finite-horizon candidate loss. For a candidate $q \in \mathcal{V}_t$ and horizon $H \geq 1$, let

$$L_H(q, t) = \sum_{\tau=t+1}^{t+H} m_{\tau}^{(q,t)} \quad (4)$$

denote the number of misses incurred over the next H requests after forcibly evicting q at time t and replaying the subsequent cache evolution under a fixed continuation rule. Here, $m_{\tau}^{(q,t)} \in \{0, 1\}$ is the miss indicator at time τ under that counterfactual trajectory. In the implementation studied here, the continuation rule is a lightweight LRU-style replay. This yields a deterministic and computationally tractable supervision target. It should be interpreted as a local proxy for the downstream harm of an eviction choice, not as an offline-optimal objective.

This candidate-level formulation is the central shift of the method. Instead of asking only whether a predictive signal should be trusted, we ask which currently cached item appears least costly to remove under the present state and context. The remainder of the method section builds on this formulation to define the learned scoring rule and then introduces practical fallback behavior for settings in which learned decisions appear unreliable.

2.2. Eviction-Value Prediction Framework

The central idea of the proposed method is to replace coarse trust-oriented control with a direct estimate of *eviction quality* at decision time. Consider a miss at time t when the cache is full, so that the candidate set is $\mathcal{V}_t = C_t$. Instead of asking only whether a predictive signal should be followed, the method assigns each candidate $q \in \mathcal{V}_t$ a score that estimates the downstream harm of evicting that candidate under the current context. The selected victim is then the candidate with the smallest predicted loss.

Let $H \geq 1$ denote a finite replay horizon. For a candidate $q \in \mathcal{V}_t$, define its horizon- H eviction loss by

$$L_H(q, t) = \sum_{\tau=t+1}^{t+H} m_{\tau}^{(q,t)}, \quad (5)$$

where $m_{\tau}^{(q,t)} \in \{0, 1\}$ is the miss indicator at future step τ under a counterfactual trajectory in which q is evicted at time t and the subsequent cache

evolution is replayed under a fixed continuation rule. In the current implementation, the default continuation rule is a lightweight LRU-style replay. This choice yields a deterministic and computationally tractable supervision signal and serves as a practical approximation rather than a claim of optimal continuation. Alternative continuation rules are treated as exploratory sensitivity directions rather than part of the main method.

The quantity in (5) is not an offline optimum. Rather, it is a local proxy for the downstream harm of an eviction choice. This is the key modeling shift of the framework. Prior learned caching methods often predict a proxy such as next arrival, reuse distance, an oracle-derived action, or a pairwise preference. Here the supervision target is tied more directly to the actual online decision: among the currently cached items, which one appears least costly to remove over a short lookahead window?

At each decision time, the framework constructs a feature vector for every candidate $q \in \mathcal{V}_t$,

$$x(q, t) \in \mathbb{R}^d, \quad (6)$$

where the features summarize the current request, the cache state, and candidate-specific predictive context. In the current repository, these features are organized into a small number of interpretable groups, including request-side features, candidate-side recency and reuse features, predictor-LRU comparison features, cache-state summary features, disagreement features, and recent-activity features. When predictive metadata are available, they are incorporated through these feature groups rather than treated as hard instructions. Table C.10 in Appendix Appendix C summarizes the feature families used by the scorer.

The learned scorer is a function

$$\hat{L}_H(q, t) = f_\theta(x(q, t)), \quad (7)$$

and the eviction decision becomes

$$q_t^* = \arg \min_{q \in \mathcal{V}_t} \hat{L}_H(q, t). \quad (8)$$

The replay target in (5) forms the basis for the optional guarded fallback mechanism described next.

2.3. Guarded Fallback Mechanism (Unvalidated Extension)

The repository also implements an optional, lightweight guard that can temporarily delegate eviction control to a conservative fallback policy when

recent online behavior suggests that the learned scorer’s decisions are locally unsafe. The guard monitors repeated *very-early-return* evictions – cases in which an item evicted by the learned scorer is requested again within a short window, treated as a signal that an item with immediate reuse value may have been removed – and, once a sliding-window count of such suspicious evictions exceeds a threshold, temporarily switches control to a conservative fallback policy (e.g., least-recently-used eviction or a marker-style policy) for a bounded duration before returning to the base learned policy. The trigger signal, window and threshold parameters, fallback duration, and fallback choice are heuristic implementation decisions, not a theorem-backed robustness guarantee [6, 7].

We treat this guard as an unvalidated implementation extension rather than as a demonstrated component of the present results: its end-to-end effect has not been measured in this study, it is not used to support any quantitative claim in this paper, and it is not part of the three empirical aims stated in Section 1.2 (see also Limitations). The full formal specification of the guard is given in Appendix B for conceptual completeness.

2.4. Algorithmic Workflow and Implementation Details

This subsection summarizes how the proposed method is instantiated in the experimental framework. The workflow is modular: trace preparation, candidate-level dataset construction, model fitting, online replay, and optional fallback control are implemented as separate components within a common replay environment. This separation keeps the learned policy, the reference baselines, and the evaluation pipeline aligned under the same execution semantics. Figure 1 provides a high-level overview of this workflow, and the five stages below summarize the guarded online decision procedure.

At a high level, the method proceeds in five stages.

1. **Trace preparation.** Public and manifest-based traces are converted into a common internal format. These processed request streams are then replayed under fixed cache capacities to construct decision points and support downstream evaluation.
2. **Candidate-level example construction.** At each full-cache miss, the current eviction candidates are enumerated and one supervised example is created per candidate. Each example contains decision-time features together with a finite-horizon replay label derived from the downstream miss count after forcibly evicting that candidate.

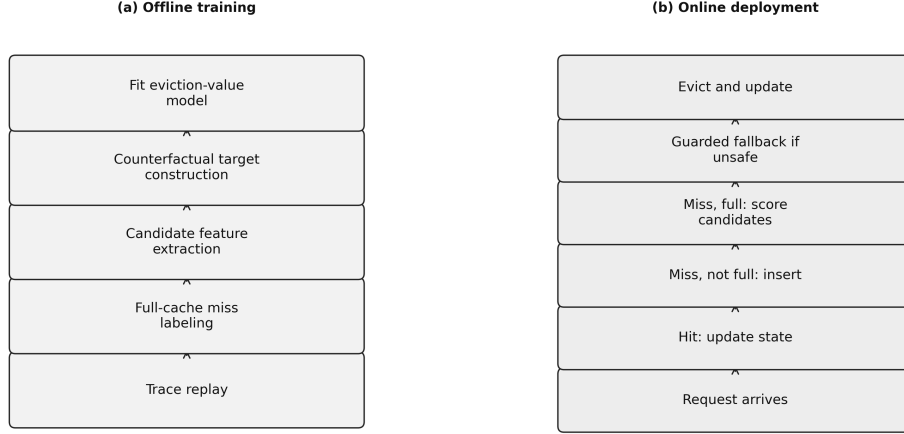


Figure 1: Conceptual overview of the eviction-value workflow. Offline replay is used to construct candidate-level training examples and finite-horizon loss targets, a supervised scorer is fitted to estimate decision-time eviction loss, and the online policy evicts the candidate with minimum predicted loss. The figure also depicts an optional, unvalidated fallback layer that can temporarily delegate control to a conservative baseline when repeated local signals indicate unsafe learned decisions; this fallback layer is not evaluated in the present study and is shown for conceptual completeness only (Section 2.3, Appendix Appendix B).

3. **Model fitting.** A supervised model is trained to predict the candidate loss target. In the present study, model selection is performed across replay horizons and model families, with nonlinear scorers emerging as the strongest configuration in the offline ablation.
4. **Online eviction replay.** During execution, the scorer evaluates all current candidates at each full-cache miss, predicts their losses, and evicts the candidate with minimum predicted loss.
5. **Optional fallback control.** When the guarded variant is enabled, the controller monitors suspicious early-return events and temporarily switches to a conservative fallback policy when recent online behavior suggests that the learned scorer is locally unsafe.

In the implementation studied here, candidate-level supervision is generated for the main eviction-value line, a scorer is trained for a chosen replay horizon, and the resulting policy is evaluated in the same simulator used for the reference baselines. The framework records aggregate metrics together with step-level logs so that downstream behavior can be analyzed jointly with fallback activity.

At decision time t , the method first builds the candidate feature matrix

$$X_t = \begin{bmatrix} x(q_1, t)^\top \\ x(q_2, t)^\top \\ \vdots \\ x(q_k, t)^\top \end{bmatrix}, \quad (9)$$

where $\{q_1, \dots, q_k\} = C_t$ is the current candidate set. The learned scorer then returns the vector of predicted losses

$$\hat{\ell}_t = \begin{bmatrix} \hat{L}_H(q_1, t) \\ \hat{L}_H(q_2, t) \\ \vdots \\ \hat{L}_H(q_k, t) \end{bmatrix}, \quad (10)$$

and the learned victim is selected by

$$q_t^* = \arg \min_{q_i \in C_t} \hat{L}_H(q_i, t). \quad (11)$$

If the optional fallback controller is active, this proposal is passed through the mechanism described in the previous subsection before the final eviction is applied. Learned policies and baselines share the same simulator and trace-loading interface.

3. Experiments and Discussion

3.1. Experimental Setup

The experiments are designed to evaluate the proposed eviction-value framework under a controlled replay protocol. All policies are executed within a common simulator using the same processed traces, cache capacities, request ordering, and trace-loading interface. This unified execution setting is important because it reduces confounding from preprocessing differences, trace-format inconsistencies, and policy-specific replay semantics. Consequently, observed differences in performance can be attributed more directly to differences in eviction behavior.

The empirical focus is the main candidate-scoring policy, `evict_value_v1`, together with a selected set of classical, predictive, and robust reference policies from the learning-augmented caching literature [3, 4, 6, 7]. These

baselines are included because they represent the most relevant practical comparison points for online paging under imperfect predictive information. Other policy families implemented in the broader codebase are treated as exploratory or contextual comparisons and are not part of the main empirical claim unless their results are explicitly reported.

The evaluation has two complementary components. The first is *offline scorer construction and selection*. In this component, candidate-level examples are generated from replayed eviction decisions, finite-horizon loss labels are constructed, and model families are compared using target-quality metrics that assess how well the learned scorer approximates the proposed supervision objective. The second is *online replay evaluation*, in which the induced policy is executed in the same simulator as the reference baselines and compared through downstream replay performance. This separation is methodologically important because strong performance on an offline proxy does not necessarily translate into the best online replacement decisions once the scorer is embedded in the full decision loop.

To isolate the eviction-choice problem, the study is conducted in the standard unweighted paging setting with fixed cache capacity and unit miss cost. The principal downstream metric is therefore the total number of cache misses over the replayed request sequence. Claims are restricted to what is directly supported by the reported evaluation outputs below.

3.2. Datasets, Baselines, and Evaluation Metrics

The empirical study is organized around a common replay interface applied to heterogeneous request traces and a baseline pool chosen to reflect the most relevant practical comparison points for learning-augmented paging. Reported results are restricted to the artifact-backed trace, capacity, and policy subset instantiated by the designated evaluation pipeline.

The baseline pool is selected to cover five comparison roles: classical paging references, CLOCK-family structural references, predictive paging references, robust learning-augmented controllers, and directly comparable learned cache-replacement systems. Classical and CLOCK-family references include least-recently used eviction, SIEVE, and FIFO-Reinsertion, which remain essential lightweight anchors for unweighted online caching [1, 2, 29]. Predictive and robust references include predictive marker, trust-and-doubt-style control, a regret-driven selective-trust gating controller (REST), and the blind-oracle/LRU combiner [3, 4, 6, 7, 5, 9]. The learned cache-replacement role adds LRB, 3L-Cache, CACHEUS, and HALP [13, 17, 18, 16], providing

the direct comparison against modern learned replacement systems reported in Section 3.4. These methods are used because they represent strong and practically relevant reference points when predictions are informative but imperfect.

Within the learned-policy family, the main method studied in this paper is `evict_value_v1`, which predicts a candidate-specific finite-horizon downstream eviction loss and evicts the candidate with minimum predicted loss. The framework also implements an optional guard that can temporarily defer to a conservative fallback policy when recent online behavior indicates locally unsafe learned decisions; this guard is an implementation safeguard evaluated only as an ablation in the present study, and we do not claim that it improves end-to-end miss ratio unless future experiments support it. Other policy families implemented in the repository, including a standalone marker-style policy, a robust follow-the-prediction combiner with marker fallback, pairwise-ranking variants, sentinel-style guards, and additional exploratory controllers, are retained for contextual analysis and ongoing research development, but they are not treated as central baselines for the present manuscript unless their results are explicitly reported in the canonical artifact bundle.

Table 1 summarizes the workload families relevant to the present study from a manuscript perspective. The table is intentionally phrased at the family level to separate source provenance from the narrower question of which canonical artifacts are currently reported. Table 2 summarizes the main policy families used in the empirical comparison.

The repository evaluates heterogeneous sources, but they are not interchangeable cache traces. BrightKite and Citi Bike are real-world check-in and trip-event streams transformed into request sequences; CloudPhysics, MetaCDN, MetaKV, and Twemcache are cache/storage-derived sources; and the Wikimedia artifact used here is a pageview-derived request proxy rather than a native CDN trace [30, 31, 32, 33, 35]. We state these distinctions explicitly in Table 1 above and treat the non-cache event streams as locality-stress workloads, not as evidence of production cache behavior.

The primary evaluation metric is the total number of cache misses incurred during online replay. We also report hit rate and, where useful for interpretation, miss deltas relative to the LRU baseline. For offline scorer analysis, we additionally use target-quality diagnostics such as regret-style error summaries, Top-1 agreement, and disagreement-oriented statistics. These auxiliary quantities help interpret the learned scorer, but downstream replay performance remains the decisive criterion.

Table 1: Trace families in the artifact-backed evaluation. The repository supports additional workloads; reported results are restricted to the canonical artifact subset.

Family	Original do-main/source	Transformation	Role
BrightKite	Location check-ins; SNAP Brightkite data [30]	Venue ID becomes the requested item; original record order is retained.	Non-cache locality stress stream.
Citi Bike	Bike-share trip histories [31]	Start-station ID becomes the requested item; original trip order is retained.	Non-cache locality stress stream.
CloudPhysics	Production block I/O collection [32]	LBN is mapped to a logical page with 4096-byte page size; read-only records are used.	Storage-derived cache workload.
MetaCDN	Meta CDN request traces via Cache-Lib/cacheMon [32]	Manifest-selected object IDs are replayed as request items.	Cache-derived workload.
MetaKV	Meta key-value cache traces via Cache-Lib/cacheMon [32]	Manifest-selected keys are replayed as request items.	Cache-derived workload.
Twemcache	Twitter production cache traces [33, 34]	Manifest-selected cluster/sample records; anonymized keys become request items.	Production cache workload.
Wikimedia	Wikimedia pageviews [35]	Pageview object IDs become request items; the evaluated artifact is a pageview-derived proxy, not a native CDN trace.	Public event-stream proxy.

Table 2: Main policy families in the empirical comparison. A diagnostic-only blind-oracle variant used for internal sanity checks is excluded from the reported comparisons. LRB, 3L-Cache, CACHEUS, and HALP are evaluated only in the matched-protocol comparison of Section 3.4, not in the Section Appendix A.1 eight-policy sweep, and are listed here for completeness of the full policy roster.

Label	Category	Role in evaluation
LRU	Classical baseline	Standard recency-based eviction reference [1].
SIEVE	Classical baseline	CLOCK-family turn-key eviction algorithm using a single FIFO queue and an advancing hand pointer over retained objects [29].
FIFO-Reinsertion	Classical baseline	CLOCK / Second-Chance family baseline that reinserts a retained survivor at the head of the queue rather than advancing a hand [29].
PredMk	Predictive baseline	Prediction-aware marker-style paging baseline [3].
BO/LRU	Combiner baseline	Blind-oracle/LRU practical combiner baseline [6].
T&D	Robust baseline	Trust-and-doubt-style controller for untrusted predictions [4].
REST	Heuristic baseline	Regret-driven selective-trust gating controller that switches between a predictor-driven eviction rule and an LRU fallback based on delayed online feedback.
LRB	Learned baseline	Belady-guided future-access learned eviction system for CDN caching; independent repository reimplementa- tion [13].
3L-Cache	Learned baseline	Low-overhead learned eviction policy using bidirectional candidate sampling; independent repository reimplementa- tion [17].
CACHEUS	Learned baseline	Learned expert-mixture eviction controller; official-source-unmodified wrap- per [18].
HALP	Learned baseline	Candidate-level preference learning from future re-access outcomes; inde- pendent reimplementa- tion, no official public code exists [16].
EV	Proposed method	Candidate-level finite-horizon eviction- value predictor.

3.3. Offline Ablation and Model Selection

We begin with the offline ablation used to select the scorer configuration for `evict_value_v1`. Table 3 and Fig. 2 summarize validation and test regret across replay horizons and model families. These results are used to characterize the learnability of the candidate-level target and to motivate the scorer configuration carried into the main policy line.

Two patterns are clear. First, the nonlinear tree-based models are consistently stronger than the linear ridge baseline. Across all three replay horizons, ridge incurs substantially larger regret than either `hist_gb` or `random_forest`. This indicates that the eviction-value target is not well captured by a simple linear approximation and benefits from a more flexible scorer.

Second, shorter replay horizons are stronger in the current artifact set. Horizon $H = 4$ gives the best validation regret for `hist_gb` and the best test regret for `random_forest`, while performance deteriorates as the horizon increases to 8 and 16. This trend is consistent with the intended role of the target: a shorter horizon appears to provide a cleaner and more learnable proxy for local eviction harm, whereas longer horizons introduce additional downstream interactions that are harder to model accurately.

The Top-1 results support the same interpretation. On both validation and test splits, the tree-based models achieve lower Top-1 error than ridge across the evaluated horizons, suggesting that nonlinear scorers preserve the candidate ordering needed for eviction decisions more effectively than a linear baseline. At the same time, the gap between `hist_gb` and `random_forest` is modest relative to their shared advantage over ridge. The main methodological conclusion is therefore about the value of nonlinear scoring and short-horizon supervision, not about one tree-based model being universally dominant.

Taken together, these ablation results support a conservative model-selection conclusion: within the current artifact-backed evidence, the candidate-level eviction-value framework is strongest with a short replay horizon and a nonlinear scorer. This evidence does not by itself establish superiority in end-to-end online replay, but it does show that the proposed target yields a meaningful and learnable signal for downstream eviction scoring.

This horizon effect reflects a specific tradeoff in label quality, distinct from the causal interpretation of online underperformance given in Section 3.6: a short horizon ties the eviction-value label closely to the immediate consequence of the scored decision, while a long horizon allows the label to absorb

Table 3: Offline ablation for `evict_value_v1` across replay horizons and model families. Lower regret is better, and lower Top-1 error is better.

Horizon	Model	Val. regret	Test regret	Val. Top-1	Test Top-1
4	hist_gb	0.0091	0.0055	0.0174	0.0316
4	random_forest	0.0134	0.0046	0.0154	0.0264
4	ridge	0.0649	0.0150	0.0503	0.0832
8	hist_gb	0.0203	0.0104	0.0157	0.0362
8	random_forest	0.0240	0.0120	0.0177	0.0267
8	ridge	0.0880	0.0384	0.0377	0.0795
16	hist_gb	0.0394	0.0196	0.0203	0.0365
16	random_forest	0.0431	0.0178	0.0157	0.0249
16	ridge	0.1166	0.0850	0.0277	0.0826

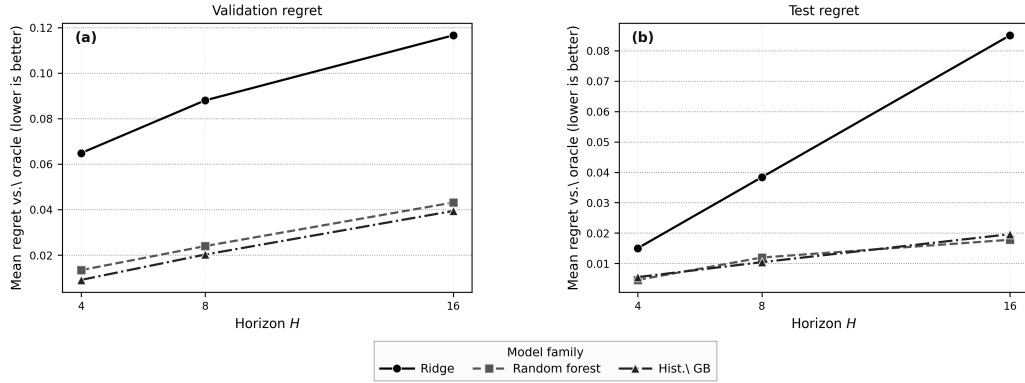


Figure 2: Offline ablation for `evict_value_v1` across replay horizons and model families. The figure summarizes validation and test regret for the configurations shown in Table 3. Lower is better.

increasing variance from the continuation policy’s own subsequent choices, effects that are downstream consequences of later decisions rather than of the eviction being scored. Beyond the regret and Top-1 metrics in Table 3, validation MAE, RMSE, and mean regret-vs-oracle also degrade monotonically as H increases from 4 to 8 to 16, for every model class evaluated. This pattern holds not only in aggregate but within every individual trace family represented in our validation sample (BrightKite, CloudPhysics, MetaCDN, Twemcache, and Wikimedia pageviews), suggesting that the advantage of shorter horizons in this framework is not specific to any one workload among those evaluated. We have not yet evaluated whether this pattern extends to the remaining trace families in our full dataset or whether it interacts with cache capacity; we flag both as open questions for future work rather than claim to have resolved them.

A historical, single-split end-to-end evaluation that predates the corrected protocol below, and that additionally covers four reference policies not included in the corrected comparison (PredMk, T&D, BO/LRU, REST), is reported in Appendix Appendix A for completeness. It is explicitly historical exploratory analysis, not primary performance evidence: the corrected, leakage-free comparison in the remainder of this section is the primary evidence for `evict_value_v1`’s comparative performance in this paper.

3.4. Matched Comparison Against Learned Cache-Replacement Baselines

This section reports a corrected, leakage-free evaluation of `evict_value_v1` and directly compares it against four learned cache-replacement systems – LRB, 3L-Cache, CACHEUS, and HALP – alongside LRU, SIEVE, and FIFO-Reinsertion, addressing both the train/test-overlap limitation disclosed in Section Appendix A.1 and the request for a direct comparison with existing learned methods, particularly LRB and 3L-Cache.

3.4.1. Corrected training protocol

`evict_value_v1` is retrained under a leave-one-family-out protocol: for each of the seven trace families in turn, the model is trained only on the other six (five for training, one for validation-only hyperparameter selection), and the held-out family contributes zero rows to training, validation, hyperparameter selection, feature selection, horizon selection, seed selection, or early stopping at any point before the model is frozen and evaluated. This removes the train/test overlap present in the model underlying Table A.8 by

construction, since a held-out family’s own requests never appear anywhere in its own model’s training pipeline.

3.4.2. Matched evaluation protocol

All eight policies compared in this section – `evict_value_v1` and the seven baselines – are evaluated on identical inputs: the same seven request traces (verified by SHA-256 hash, not filename), cache capacities 32, 64, and 128 under identical object-slot capacity and unit-object-size semantics, an identical history prefix [0, 10,000), an identical scored suffix [10,000, 50,000) (40,000 scored requests), and identical hit/miss accounting. We emphasize that *matched evaluation protocol* refers only to these evaluation-side inputs and does not imply identical training or adaptation procedures: LRB, 3L-Cache, and CACHEUS are online/adaptive baselines that learn continuously from their own in-trace stream with no cross-family training corpus at all; HALP is trained offline but only on each trace’s own history prefix; LRU, SIEVE, and FIFO-Reinsertion are parameter-free; and `evict_value_v1` is trained offline with the leave-one-family-out procedure above. No baseline in this comparison, including `evict_value_v1`, has access to future request information or to another family’s data at any point.

3.4.3. Results

Table 4 reports, for each baseline, the mean miss ratio over the 21 matched (family, capacity) cells, the number of cells `evict_value_v1` wins, loses, and ties against that baseline, and the relative mean difference in miss ratio. `evict_value_v1`’s own mean primary miss ratio across all 21 cells is 0.700463.

Under this matched comparison, `evict_value_v1` does not outperform any of the seven baselines: it loses on a clear majority of the 21 matched cells against every one (13–17 losses, 1–5 wins, 3–4 ties), with a positive (worse) mean relative miss-ratio difference ranging from +1.13% (3L-Cache) to +4.12% (LRU). This directly answers the request for a fair comparison against LRB and 3L-Cache: the comparison shows that `evict_value_v1` does not offer an empirical advantage over either under matched evaluation.

3.4.4. Implementation-fidelity caveats

LRB and 3L-Cache are independent repository reimplementations of their respective published algorithms, evaluated under the exact matched protocol above; 3L-Cache uses a fixed default batch size not re-tuned for this study,

Table 4: Matched-protocol comparison of `evict_value_v1` against learned and classical cache-replacement baselines, 21 (family, capacity) cells per baseline. Positive relative difference indicates `evict_value_v1` has a higher (worse) mean miss ratio than the baseline. LRB, 3L-Cache, CACHEUS, and HALP are learned baselines; LRU, SIEVE, and FIFO-Reinsertion are classical.

Baseline	Baseline mean	EV wins	Baseline wins	Ties	Rel. diff.
LRB	0.6877	5	13	3	+1.85%
3L-Cache	0.6926	5	13	3	+1.13%
CACHEUS	0.6756	3	15	3	+3.68%
HALP	0.6762	1	17	3	+3.59%
LRU	0.6728	1	16	4	+4.12%
SIEVE	0.6875	4	14	3	+1.89%
FIFO-Reinsertion	0.6733	2	16	3	+4.04%

which we record as a material implementation caveat on its comparison. CACHEUS is executed via an official-source-unmodified wrapper around the authors’ own released decision engine, with a provenance caveat that the external upstream clone is not currently live-verifiable in the present verification environment; this is a source-authenticity caveat, not an evaluation-input caveat. HALP is an independent reimplementaion of the published method, since no official public HALP implementation exists; we treat it as a lower-fidelity supporting comparison rather than a primary result, consistent with Reviewer #3’s specific request for differentiation from HALP rather than an official empirical baseline. None of these caveats affect the matched-evaluation-protocol dimensions in Table 4; they affect only how faithfully each reimplementaion reproduces its source algorithm’s exact behavior.

3.5. Direct Comparison Against Alternative Supervision Objectives

Section 2.2 motivates the finite-horizon eviction-loss target on the grounds that it is defined directly at the eviction decision, in contrast to indirect proxies such as reuse distance, next-arrival time, or pairwise candidate preference. This premise is testable, and we test it directly rather than argue for it only conceptually.

Using the same leave-one-family-out training and matched evaluation protocol as Section 3.4, we train and evaluate three alternative candidate-level supervision objectives in place of eviction-loss, holding the horizon, model family, feature set, and every other pipeline component fixed: a reuse-distance target (candidate-specific time to next reuse), a next-arrival target

(candidate-specific time to the next request for that item), and a pairwise-preference target (candidate-specific relative ranking rather than an absolute value). Table 5 reports total misses over the 21 matched primary cells for each of the four objectives, including the proposed eviction-loss target.

Table 5: Supervision-objective comparison under the matched leave-one-family-out protocol, 21 primary cells, total misses (lower is better).

Objective	Total misses	Mean miss ratio
Eviction loss (proposed)	601,569	0.7162
Next arrival	573,059	0.6822
Reuse distance	571,456	0.6803
Pairwise preference	565,127	0.6728

The proposed eviction-loss instantiation of `evict_value_v1` is the worst of the four full-pipeline alternatives, and it is the single worst objective within every one of the seven trace families individually (in the degenerate Wikimedia family, all four objectives tie at full misses), not merely on average. This contradicts the premise, stated in Section 2.2, that decision-aligned finite-horizon eviction-loss supervision is inherently better matched to the eviction decision than these indirect alternatives *in the deployed `evict_value_v1` pipeline*. We report that pipeline result plainly.

That comparison, however, does not isolate the training objective as the sole causal factor: other pipeline differences can still co-vary with the label. To test the narrower hypothesis that the eviction-loss *training objective itself* is responsible for the poor learned-policy performance, we run a matched common-model control on the same 21 (family, capacity) cells. Architecture, features, folds, windows, and seed are held fixed; only the supervision objective changes, and the pairwise objective uses a corrected orientation that trains on the intended preference label rather than candidate-identifier order. Table 6 reports the result.

Under this matched control, eviction-loss is nominally best by total misses and by macro mean miss ratio; pairwise preference is second by totals ($\approx 0.9\%$ behind) but strongest by per-cell rank. Eviction-loss is not materially worse than the alternatives. The matched experiment therefore does *not* support the hypothesis that the eviction-loss training objective itself explains the poor performance of the full learned policy. The Table 5 pipeline ranking remains a valid description of `evict_value_v1` as deployed; it should not be read as an isolated objective-causality result. Section 3.6 investigates

Table 6: Matched common-model objective control (V2): identical folds, windows, features, architecture, and seed; only the training objective changes. 21 primary cells, total misses (lower is better).

Objective	Total misses	Mean miss ratio
Eviction loss	571,976	0.6809
Pairwise preference	577,339	0.6873
Reuse distance	615,850	0.7332
Next arrival	627,392	0.7469

the remaining, stronger supported diagnosis: the horizon-4 target is highly action-underdetermined, so downstream selection among many exact minimizers dominates online behavior.

3.6. Mechanistic Diagnosis of the Eviction-Loss Target

Sections 3.4 and 3.5 establish that `evict_value_v1` underperforms both external baselines and, in its full pipeline, alternative supervision objectives, while the matched common-model control in Table 6 shows that this is not explained by the eviction-loss training objective alone. This section asks *why*, using targeted diagnostics run under the same matched protocol and the same 21 (family, capacity) cells. We organize the diagnosis as a sequence of questions, each answered by a specific measurement rather than by argument.

Is inaccurate prediction the cause? If the learned scorer poorly approximated the eviction-loss target, that alone would explain poor downstream performance. We test this by comparing the learned policy’s decisions against an *exact* solver for the same target (a per-decision oracle that computes the true horizon-4 eviction loss for every candidate directly, rather than a learned approximation). Across the 21 cells, the learned scorer’s decisions agree with the exact target’s optimal candidate set in 97.53% of decisions (macro mean set-aware agreement across cells), with a mean target regret of 0.0247 and a positive-regret fraction of 0.0247; total learned misses (601,569) are within 6.4% of the LRU reference (565,126) despite this near-perfect target agreement. This disfavors gross model-fitting failure as the primary explanation: the learned scorer is doing what it is asked to do.

Is the target itself useful, even if perfectly optimized? We first replay the exact horizon-4 oracle under a deterministic tie convention (minimum candidate identifier among exact minimizers), rather than a learned approximation of it. Across the same 21 cells, that deterministic oracle has a

mean miss ratio of 0.770090, compared to 0.672769 for LRU – it loses to LRU on 18 of 21 cells, ties on 3, and never wins, with a mean cell-level disadvantage of +0.0973 and 81,750 additional total misses (646,876 versus 565,126). This shows that the *deterministic target-plus-tie-resolution instantiation* is not a good online eviction policy. It is not, by itself, a statement about every policy that could resolve the same optimal sets differently.

A follow-up tie-aware control therefore varies only how exact minimizers are chosen, using the same 21 cells and the same exact H4 minimizer sets. Every measured non-LRU oracle decision has multiple optimal actions (`fraction_tied_decisions` = 1.0 on all 168 tie-aware rows; mean fraction of all-candidates-tied decisions ≈ 0.649 ; mean optimal-set fraction ≈ 0.991 ; the three Wikimedia cells are fully degenerate). Choosing the LRU-most candidate among exact minimizers never loses to LRU: it wins on 16 of 21 cells, ties on 5, and incurs 413 fewer total misses than LRU (564,713 versus 565,126). Choosing the MRU-most minimizer reproduces a deficit similar to the deterministic oracle (0 wins / 3 ties / 18 losses; +89,135 total misses). Uniform random choice among minimizers, over five fixed seeds, still usually loses to LRU (7 wins / 15 ties / 83 losses over 105 rows) but usually beats the deterministic oracle (90 wins / 15 ties / 0 losses). The previously reported deterministic exact-oracle deficit versus LRU is therefore a tie-breaking confound. It does *not* establish that the exact H4 target intrinsically performs worse than LRU. What the data do establish is that the target is highly action-underdetermined, so downstream selection among many exact minimizers can change the online result from a large deficit to a small advantage over LRU.

Why is the target so underdetermined? We audit the target’s decision-level structure independently of any one tie rule. Across the same 21 cells, no decision ever has a unique optimal eviction candidate (unique-winner fraction 0.0); every decision has multiple tied-optimal candidates (multiple-optimum fraction 1.0); in 76.4% of decisions in the strict-preference diagnostic every resident candidate is exactly tied for optimal; and on average 99.5% of resident candidates fall within the optimal set (mean optimal-set fraction 0.9949). These figures describe the same qualitative degeneracy as the tie-aware replay above. The horizon-4 target almost never provides a strict, informative preference among the cached candidates, so the specific eviction choice a learned or exact policy makes among the many tied-optimal candidates is effectively unconstrained by the target, yet still determines downstream performance. A companion diagnostic quantifies an underly-

ing observability limit: across 41,615,776 candidate-residency observations at 565,126 decision points, the probability that an item’s true reuse time exceeds the 4-step horizon is 0.9939 overall and 0.9793 even conditioned on the item eventually being reused. The horizon-4 window observes only a small fraction of the reuse behavior that determines whether an eviction was actually costly, which is consistent with – and offers a mechanistic explanation for – the observed target degeneracy: most candidates’ true consequences fall outside the window the target can see, so the target frequently cannot distinguish them.

Summary. A separate learning-curve check across five training-set fractions from 1% to 50% of the available candidate-level examples shows no material improvement in target-quality metrics with additional data, disfavoring insufficient training data (Section 3.3’s offline ablation already selects the best-performing configuration within this data regime). Taken together with the agreement, oracle, degeneracy, and tie-aware diagnostics above, these results disfavor both insufficient training data and gross model-fitting failure. They also fail to support the claim that the eviction-loss training objective itself is the culprit (Table 6). The strongest supported diagnosis is that the horizon-4 target is highly action-underdetermined, so selection and deployment behavior inside a large optimal set matter substantially. Horizon truncation relative to typical reuse timescales is a consistent mechanistic background for that degeneracy. This does not, by itself, prove that tie degeneracy explains every aspect of learned-policy underperformance. Section 3.7 next examines a secondary, partial contributor: the mismatch between the label-construction continuation rule and the policy’s own deployed trajectory.

3.7. Continuation-Mismatch and Distribution-Shift Ablations

Section 3.6 establishes action-underdetermined (tie-degenerate) supervision, and the sensitivity of exact-oracle replay to how minimizers are resolved, as the strongest supported diagnosis. This section reports two further controlled experiments testing whether a learnable, decision-aligned offline target would translate into a policy that underperforms strong baselines online because of the specific continuation-rule assumption used to build the supervision target, or because of a broader mismatch between the training-state distribution and the deployment-time trajectory. Both are controlled causal tests of hypotheses that were speculative in an earlier version of this study: that the eviction-loss label’s fixed LRU-style continuation assumption diverges from the policy’s own recursive, non-LRU deployment trajectory, and

that this divergence contributes to the observed gap.

We test the continuation-mismatch hypothesis directly with a three-arm causal ablation under the leave-one-family-out protocol, isolating exactly one change between arms. *C0* replays LRU end-to-end (the reference). *C1* trains a policy π_1 on labels built under LRU continuation (matching Eq. (5) as originally specified) and deploys π_1 recursively. *C2* trains a second policy π_2 on labels built under a corrected continuation rule – the frozen π_1 ’s own decisions, rather than LRU – and deploys π_2 recursively; this is the only change between *C1* and *C2*. Across the 21 (family, capacity) cells, *C2* improves over *C1* in 13 cells, ties in 3 (the degenerate Wikimedia cells), and worsens in 5, with a macro mean miss-ratio improvement of -0.0102 ; aggregate misses move from 601,569 (*C1*) to 592,970 (*C2*), against a 565,126-miss LRU reference (*C0*). The improvement is not uniform: one cell, BrightKite at capacity 32, regresses sharply ($+0.2433$). The present experiments do not isolate the mechanism behind this specific reversal, and we do not speculate about it here; it remains an important counterexample that by itself prevents a claim of uniform benefit. We therefore classify the continuation-mismatch hypothesis as *partially supported and regime-dependent*: correcting the continuation-label mismatch helps in most but not all family/capacity combinations, and is a real but secondary and non-uniform contributor to the gap documented in Sections 3.4–3.6, not a full explanation of it.

A related but distinct question is whether the broader training-state distribution – not just the continuation label itself – diverging from the deployment-time trajectory is the operative mechanism, and whether correcting it helps. We test this with a one-step DAgger-style intervention that retrains on a mixture of on-policy and off-policy states to reduce a generic measured state-visitation shift index, holding everything else fixed. Across the same 21 cells, this intervention reduces the measured shift index in 16 of 21 cells, yet online misses improve in only 2, tie in 3, and worsen in 16, with a macro mean miss-ratio degradation of $+0.0094$ (aggregate misses $591,604 \rightarrow 599,537$). In 13 of the 18 informative (non-degenerate) cells, the shift metric improves while misses simultaneously worsen. We report this as a clear negative result for this specific intervention: reducing this generic, DAgger-style measure of state-distribution shift does not, by itself, improve online performance, even though the shift-reduction mechanism functions as intended on its own metric. This does not mean distribution shift does not exist as a phenomenon; it means that this particular corrective intervention, targeting a generic shift measure rather than the decision-relevant continua-

tion label specifically, is not an effective remedy in the tested setting.

3.8. Discussion and Synthesis

The present results are most informative when interpreted through the central modeling choice of the paper: the proposed method is organized around a decision-aligned supervision target for candidate-level eviction scoring rather than around a global trust-calibration rule. This distinction is important because cache replacement is inherently a local comparative decision. At a full-cache miss, the policy must decide which currently cached item is least harmful to remove, not merely whether predictive information should be trusted in the abstract. From this perspective, the most direct question is not whether the advice appears accurate in isolation, but whether the learned target helps rank the available eviction candidates in a way that improves the actual online choice.

Viewed in this way, the offline ablation indicates that replay-based candidate supervision provides a viable route for translating predictive context into eviction decisions; tree-based scorers consistently outperform the linear ridge baseline across replay horizons, and shorter replay horizons yield the strongest regret values within the current artifact set, consistent with the interpretation that longer horizons incorporate harder-to-model downstream interactions (Section 3.3). A candidate-level downstream-loss target of this kind remains a proxy, but it is defined directly at the decision point, which gives it decision alignment by construction; whether that alignment is also informative and efficient enough for competitive online use is exactly the question the remainder of this paper tests.

At the same time, the interpretation of these results should remain cautious, and the end-to-end evaluation in Appendix A sharpens rather than removes that caution. The replay-based loss studied here is finite-horizon and local rather than globally optimal, and the offline scorer-selection ablation does not by itself establish superiority in full end-to-end online replay. The explicit end-to-end evidence we report directly answers that question, and the answer is negative: across all three evaluated capacities, `evict_value_v1` incurs more replay misses than LRU, SIEVE, and FIFO-Reinsertion, with the gap widening sharply and non-monotonically at capacity 128. Likewise, the broader framework includes guarded variants whose end-to-end effect remains unmeasured in the present study (see Limitations). The formulation is plausible and directly aligned with the eviction decision by construction; the experiments in this paper test whether that

structural alignment is, by itself, sufficient for competitive online behavior. Current evidence does not establish competitive online utility: it does not support a claim of downstream competitiveness, robustness across workload regimes, or effective fallback control in deployment, and the end-to-end evidence weighs against competitiveness at every capacity evaluated so far.

Taken together, the artifact-backed evidence supports a clear but negative conclusion about the present instantiation of `evict_value_v1`, together with a mechanistically informed – and now more carefully bounded – account of why. Finite-horizon candidate-level scoring, as a general framework for translating predictive context into eviction decisions, is directly decision-aligned by construction and was learnable in the offline ablation, where short-horizon nonlinear scorers were the strongest configuration for the eviction-loss target specifically (Section 3.3). The full `evict_value_v1` pipeline does not outperform any of seven classical or learned baselines (Section 3.4), and in that pipeline eviction-loss is the worst of four supervision objectives (Table 5). A matched common-model control, however, shows that this pipeline ranking is not explained by the eviction-loss training objective itself (Table 6). The strongest supported diagnosis is instead that the horizon-4 target is highly action-underdetermined: every measured non-LRU exact-oracle decision has multiple optimal actions, so downstream selection among minimizers can change online performance from a large deterministic-oracle deficit versus LRU to a small LRU-within-minima advantage (Section 3.6). Continuation-label mismatch contributes a real but partial, regime-dependent secondary effect (Section 3.7), and generic distribution-shift correction offers no remedy on its own. We characterize `evict_value_v1`, in its present form, as a decision-aligned supervision study whose deployed policy is unsuccessful and whose exact target is informative only after a secondary rule is supplied, rather than as a practically superior online policy. LRU, SIEVE, FIFO-Reinsertion, LRB, 3L-Cache, CACHEUS, and HALP all remain baselines the proposed method does not currently outperform under matched evaluation. Future work on this framework should treat horizon design, target informativeness, and explicit tie resolution as primary design questions, not assume that model capacity, training data volume, or a change of training objective alone will solve the problem.

3.9. Overhead and Scalability

The proposed method has two distinct computational costs: an offline label-construction cost incurred once per dataset, and an online per-decision

cost incurred on every full-cache miss during replay or deployment. We report both below, separating measured wall-clock evidence from complexity analysis.

Constructing the finite-horizon eviction-value dataset used in this paper required simulating a fixed continuation policy forward from every eviction decision considered, which is inherently more expensive than constructing a supervision target that can be read directly from the trace (e.g., next-arrival time); this offline construction step took approximately 10.3 hours of wall-clock time and produced 96 GB across 662 shard files. Model training itself is comparatively inexpensive, since it operates on a bounded sample of the constructed dataset rather than on the full label set directly; fitting all evaluated horizon/model-class configurations together took approximately 7–8 minutes.

At each full-cache miss, `evict_value_v1` constructs a feature vector for every one of the k candidates currently resident in the cache and scores each with the trained model, giving $O(k)$ model-inference calls per miss, where k is the cache capacity. This contrasts with the $O(1)$ -per-miss classical baselines in our comparison: LRU evicts via a single recency-ordered removal, and the CLOCK-family baselines in our pool, SIEVE and FIFO-Reinsertion, evict via an amortized $O(1)$ hand-scan or reinsertion-scan respectively, with no per-candidate model inference. This is the expected and inherent cost of candidate-level learned scoring relative to structural eviction rules, and it grows with cache capacity rather than remaining constant.

This complexity comparison is verified directly against our own implementations’ source code rather than taken solely from the respective baselines’ publications. We have since replaced our earlier single-machine, single-run local benchmark with a controlled, repeated-measurement timing campaign executed on a dedicated compute node: 7 trace families $\times$ 3 capacities $\times$ 4 policies $\times$ 5 repetitions (420 total timed runs), covering LRU, FIFO-Reinsertion, SIEVE, and a causal reimplement of HALP. Table 7 reports the mean, median, and 95% confidence interval of the per-request wall-clock cost for each policy across all 105 measurements per policy.

LRU, FIFO-Reinsertion, and SIEVE remain single-digit-microsecond-per-request policies under this controlled measurement, confirming the $O(1)$ complexity-class comparison above with repeated-measurement statistical support rather than a single run. HALP, as a learned policy with online candidate-preference inference, is substantially slower ($186\times$ LRU’s mean), consistent with the $O(k)$ -per-miss cost of candidate-level model inference.

Table 7: Controlled timing campaign: mean/median wall-clock cost per request (μs), 7 families $\times$ 3 capacities $\times$ 5 repetitions per policy ($n = 105$), with 95% confidence intervals and slowdown relative to LRU.

Policy	Mean (μs)	Median (μs)	95% CI	Slowdown
LRU	4.68	4.59	[4.45, 4.91]	1.00 $\times$
FIFO-Reinsertion	5.17	5.14	[4.96, 5.38]	1.10 $\times$
SIEVE	9.52	8.85	[8.91, 10.14]	2.03 $\times$
HALP (causal)	870.66	920.14	[773.47, 967.86]	186.02 $\times$

`evict_value_v1` was not included in this controlled 5-repetition campaign: its own runtime is substantially higher still, and a full 5-repetition run across all 21 cells was judged computationally infeasible within this study’s compute budget. In its place, we report `evict_value_v1`’s single, non-repeated wall-clock runtime as recorded during the corrected held-out evaluation of Section 3.4: for example, 35,290 μs per scored request on the BrightKite trace at capacity 32, roughly four orders of magnitude slower than LRU and still substantially slower than HALP. Because this is a single measurement with no repetition variance, we do not report a confidence interval for it, and we do not place it in Table 7 alongside the controlled four-policy campaign; the two should not be treated as directly statistically comparable. Both pieces of evidence support the same qualitative conclusion: the current implementation is not competitive with lightweight baselines on a wall-clock basis, and this cost, together with the negative miss-ratio comparisons above, argues against treating the current instantiation as a production-ready replacement for lightweight cache policies.

3.10. Practical Significance

Given the negative miss-ratio comparison in Section 3.4 and the timing evidence in Section 3.9, we state directly what practical value, if any, the present study offers, rather than leaving this question implicit. As currently instantiated, `evict_value_v1` is not a candidate for replacing LRU, SIEVE, FIFO-Reinsertion, or the evaluated learned baselines in latency- or throughput-sensitive deployments. We do not identify a deployment scenario in which this specific cost/performance combination is currently justified, and we do not claim one.

The value of the present study is instead diagnostic and methodological: it provides a controlled empirical test of a specific supervision-target

design (finite-horizon, candidate-level eviction loss) that is representative of a broader family of approaches in learning-augmented caching, and it isolates, through Sections 3.5–3.8, which aspects of that design are, and are not, responsible for its failure to outperform simpler alternatives. This kind of evidence is useful precisely because it is negative and mechanistically constrained rather than merely negative: it identifies action-underdetermined supervision and tie-sensitive exact-oracle replay, not the eviction-loss training objective, model capacity, training data volume, or gross prediction error, as the primary supported barrier, which directly informs what a future redesign of this target family would need to change (Section 4.4). We see two narrower settings where the present framework’s cost profile could plausibly become acceptable in future work, not in its current form: offline or batch-oriented research and analysis pipelines where decision quality, not per-decision latency, is the binding constraint and scoring cost can be amortized outside the request path; and as a supervision-objective research tool for evaluating alternative finite-horizon targets before any online deployment is considered, which is in fact how it is used within this paper (Section 3.5). We do not present either as a validated production use case.

Recent cache systems illustrate why this distinction matters. Rather than placing a large learned scorer on every eviction path, recent designs reduce cost through candidate filtering at the tail (MAT), group-level inference that amortizes signals across objects (GL-Cache), cache-level parameter learning separated from the data plane (LAH/S4-FIFO), or carefully structured low-overhead sampling and characterization (3L-Cache and Merlin) [24, 25, 26, 17, 27]. Strong non-ML designs such as S3-FIFO and SIEVE provide a complementary reminder that simple queue structures can deliver useful efficiency and scalability without learned per-candidate inference [28, 29]. We therefore frame the practical lesson narrowly: a learned cache replacement design must jointly provide decision information, favorable online miss behavior, and acceptable hot-path cost. The present implementation demonstrates none of those three conditions strongly enough for deployment, but it provides a controlled test of how a seemingly decision-aligned target can fail them.

4. Conclusions and Future Research

4.1. Summary of Findings

This paper studied learning-augmented caching through a candidate-level view of the eviction decision. Rather than treating prediction primarily as advice to be trusted or ignored at a global level, the paper formulated each full-cache miss as a structured comparison among the items currently in cache and developed a method that predicts the finite-horizon downstream loss associated with evicting each candidate.

The offline ablation shows that the proposed eviction-value target is learnable in isolation, that nonlinear scorers are substantially stronger than a linear ridge baseline, and that shorter replay horizons provide the strongest results within the present artifact set. The overall framework also includes a lightweight, optional fallback layer that can defer to a conservative baseline when recent local behavior appears unsafe; Section 4.3 discusses why this remains a design extension rather than a validated empirical contribution in the present study.

The remaining findings are cautionary and, taken together, negative for the present `evict_value_v1` instantiation, but they are directly and mechanistically explained rather than left as an unexplained empirical gap. End-to-end online replay evaluation, both under the original single-split protocol (capacities 32, 64, and 128; Section Appendix A.1) and under a corrected, leakage-free protocol matched against seven classical and learned baselines including LRB and 3L-Cache (Section 3.4), shows that `evict_value_v1` does not outperform LRU, SIEVE, FIFO-Reinsertion, LRB, 3L-Cache, CACHEUS, or HALP. A full-pipeline ablation against three alternative finite-horizon supervision objectives (Section 3.5) shows that the eviction-loss instantiation is the worst or tied-worst of the four in every trace family, while a matched common-model control shows that this ranking is not explained by the eviction-loss training objective itself. A mechanistic diagnosis (Section 3.6) finds severe target degeneracy at the tested horizon and shows that the deterministic exact-oracle deficit versus LRU is a tie-breaking confound: LRU-within-minima never loses to LRU. A controlled causal ablation (Section 3.7) finds that mismatch between the label-construction continuation rule and the policy’s deployed trajectory is a real but partial, family-dependent secondary contributor, and a companion distribution-shift-correction experiment shows that reducing a generic measure of this shift does not by itself improve online performance. Finally, a controlled timing

campaign shows the learned policy’s per-decision cost is orders of magnitude higher than the lightweight baselines (Section 3.9). Capacity-256 evaluation and guarded-deployment evidence remain left to future work.

4.2. Implications of the Proposed Approach

The proposed approach, and its negative outcome, both have implications for learning-augmented cache design. The first is methodological: in cache replacement, the supervision target can be as important as the predictive signal itself, but proximity to the eviction decision is not, by itself, sufficient for a target to be effective. The eviction-loss target studied here is defined directly at the candidate level, yet the deployed `evict_value_v1` pipeline is outperformed by less directly loss-aligned alternatives (Table 5) and, more importantly, the exact target is highly action-underdetermined at the tested horizon (Section 3.6). A matched common-model control shows that this is not because the eviction-loss training objective is itself a uniquely harmful label (Table 6). The present study therefore suggests a more specific design lesson than “define the target close to the decision”: a finite-horizon target must also be checked for decision-level informativeness (whether it actually distinguishes candidates), for how ties among minimizers are resolved, and for horizon adequacy relative to the workload’s characteristic reuse timescale. In the terms introduced in Section 1.2, decision alignment alone is not sufficient; decision informativeness and deployment efficiency (Sections 3.9–3.10) must also hold. This does not, by itself, imply that candidate-level or decision-aligned learning is unworkable for cache replacement in general: the negative result is a property of this specific target, horizon, and deployed pipeline, not a general failure of candidate-level supervision.

A second implication is that robust learning-augmented caching need not be expressed only through global trust calibration or expert switching: in the present implementation, predictive information is incorporated through candidate-level scoring rather than through a global trust signal. The code-base also includes an optional guard-style fallback extension (Section 2.3, Appendix Appendix B); it is not used to support any quantitative claim in this revision, so the main evidence for this implication concerns the candidate-level scoring formulation and its end-to-end replay behavior, not a validated fallback mechanism.

A third implication concerns evaluation methodology. For cache-control methods, it is not enough to assess the quality of an intermediate predictive signal in isolation. What matters is whether the induced eviction decisions

improve downstream replay behavior under a controlled simulator, with clear separation between offline target quality and online policy quality.

4.3. Limitations

The results of this study should be interpreted in light of several limitations. First, the main method is evaluated in the standard unweighted paging setting with fixed cache capacity and unit miss cost. This setting is well suited to isolating the eviction-choice problem and maintaining comparability with established reference policies, but it does not by itself establish effectiveness for weighted paging, variable object sizes, retrieval costs, byte-oriented objectives, or broader general-caching settings.

Second, the proposed supervision target is intentionally local and finite-horizon. The replay-based loss is designed as a tractable approximation to downstream eviction harm rather than as a globally optimal objective. As a result, the learned scorer is trained to predict a local decision proxy rather than an offline-optimal replacement rule. This design is central to the practicality of the method, but it also limits the strength of any claim about long-range optimality.

Third, the end-to-end evidence against `evict_value_v1` is now extensive and, unlike in an earlier version of this study, mechanistically explained rather than merely observed. The single-split evaluation in Section Appendix A.1 shows negative results at every evaluated capacity but relies on a model with documented train/test overlap (disclosed in that section); the corrected, leakage-free evaluation in Section 3.4 confirms the same qualitative conclusion against seven classical and learned baselines. The full-pipeline objective ablation in Section 3.5 shows the eviction-loss instantiation of `evict_value_v1` is outperformed by alternative targets in that pipeline, but the matched common-model control shows that this is not because the eviction-loss training objective itself is uniquely harmful. Sections 3.6 and 3.8 identify severe target degeneracy – equivalently, a highly action-underdetermined exact target – as the leading supported diagnosis, and show that deterministic exact-oracle replay is tie-sensitive: it must not be read as proof that the exact target intrinsically loses to LRU. Continuation-label mismatch remains a real but partial, family-dependent secondary contributor (13 of 21 cells improve under a corrected continuation rule, but one cell regresses sharply), while generic distribution-shift correction offers no remedy on its own (a DAgger-style intervention reduces the measured shift

metric in 16 of 21 cells while online misses worsen in 16 of 21). These experiments constrain interpretation rather than proving one complete causal mechanism: they do not isolate every downstream cause of learned-policy underperformance, and they remain bounded by the specific horizon ($H = 4$), continuation-correction method (one-step, frozen-policy relabeling), and shift-reduction method (one-step DAgger) tested. A different horizon, a multi-round continuation correction, or a decision-targeted (rather than generic) shift-reduction method are not ruled out and are left to future work (Section 4.4). Capacity 256 has not been evaluated end-to-end under either protocol.

Fourth, the paper is methodological and empirical rather than theorem-centered. We do not provide a new competitive-ratio guarantee for the learned policy, and the fallback mechanism is introduced as a practical control layer rather than as a formally proved safeguard. The contribution should therefore be interpreted as a decision-aligned learning framework for cache replacement, not as a replacement for the classical worst-case guarantees available in online algorithms.

Fifth, empirical conclusions in learning-augmented caching remain sensitive to workload characteristics, cache capacities, and replay protocol choices. The seven families are deliberately heterogeneous, but BrightKite and Citi Bike are non-cache event streams transformed into request sequences, Wikimedia is a pageview-derived proxy, and the cache/storage-derived families are themselves sampled or transformed by their source formats. The 50,000-request windows and unit-size paging abstraction also do not reproduce the scale, byte costs, or admission semantics of production caches. Baseline implementation fidelity is not uniform: LRB and 3L-Cache are documented independent reimplementations, CACHEUS uses the authors’ decision engine through an adapter, and HALP is a lower-fidelity supporting adaptation. Comparisons are descriptive paired replay results over the 21 family-capacity cells, not inferential claims about a population of independent traces; the learned evaluations also use fixed seeds. The current results should therefore be read as evidence bearing directly on the tested target, horizon, workload transformations, and evaluated implementations, rather than as a claim ruling out every possible finite-horizon candidate-level formulation.

Sixth, the baseline-specific implementation-fidelity caveats for the matched-protocol comparison (batch-size, provenance, and reimplementation caveats for 3L-Cache, CACHEUS, and HALP respectively) are detailed once, in full, in Section 3.4’s implementation-fidelity caveats; they are separate from, and

do not affect, the matched-evaluation-protocol dimensions themselves. Relatedly, `evict_value_v1`'s reported runtime (Section 3.9) is a single, non-repeated measurement, not part of the controlled 5-repetition timing campaign covering LRU, FIFO-Reinsertion, SIEVE, and HALP; a directly comparable repeated-measurement timing campaign for `evict_value_v1` was judged computationally infeasible within this study's compute budget given its substantially higher per-decision cost, and should not be conflated with the four-policy controlled campaign.

Seventh, this revision is the work of a single author, and the validation reported here cannot substitute for independent replication by additional authors or institutions. To mitigate, but not eliminate, this risk, the revision relies on repository-level audit trails, schema and replay-artifact validation checks, and explicit negative-result reporting in place of multi-author cross-checking.

Finally, the fallback variants introduce additional design choices, including the early-return window, trigger threshold, monitoring interval, fallback duration, and fallback policy. These choices are operationally interpretable and easy to implement, but they remain heuristic parameters rather than theoretically derived constants. Their value may therefore depend substantially on workload regime and deployment conditions.

Taken together, these limitations define the intended scope of the paper. The study makes a focused, decision-aligned empirical case for testing candidate-level finite-horizon eviction-value prediction as a supervision-target design in learning-augmented caching – a test whose result is negative, as detailed above – while leaving broader cost models, stronger guarantees, and more extensive regime analysis to future work.

4.4. Future Research Directions

Several research directions follow from this study's negative but mechanistically constrained result. First, and most directly motivated by Section 3.6, future work should treat explicit tie resolution and target informativeness as first-class design questions: adaptive or longer replay horizons, terminal-value correction that extends the label beyond the truncation point, or uncertainty-aware and soft decision rules that do not require a strict preference to exist among tied-optimal candidates. The LRU-within-minima result shows that a recency secondary rule among exact minimizers can already change online behavior substantially; it does not, by itself, yield a competitive learned policy. Second, the continuation-mismatch finding in Section 3.7 is

regime-dependent and based on a single-round correction; multi-round or iterative continuation correction, and a decision-targeted (rather than generic) distribution-shift correction, are natural next steps given that the tested one-step DAgger-style intervention did not itself improve performance. Third, the candidate-centric framework should be extended beyond unweighted paging to weighted and general-caching models with variable object sizes and heterogeneous miss costs. Fourth, ranking-based or other alternative supervision remains an open design axis, but it should not be motivated by the disproven claim that the eviction-loss training objective is uniquely harmful under a matched model. Fifth, the optional guarded controller remains heuristic; future work should develop validated fallback policies, including a confidence- or uncertainty-based trigger informed by the target-degeneracy findings above, and evaluate them together with broader capacity sweeps including capacity 256. Sixth, more efficient inference (e.g., approximate or selectively invoked candidate scoring) would be required before the overhead documented in Section 3.9 could be reduced enough to reconsider the practical framing in Section 3.10. Finally, where possible, partial theoretical guarantees for restricted candidate-scoring or guarded deployment settings remain open. We list these as concrete, evidence-motivated directions rather than as completed work; none of the interventions above, other than the specific tie-aware and common-model controls already reported, has been evaluated as a full redesigned policy in this study.

Acknowledgements

The author thanks Professor Ioannis Koutis for his guidance and support throughout this work. The author also acknowledges the use of the Wulver high-performance computing system at the New Jersey Institute of Technology for part of the experimental evaluation.

Funding

The author received no specific funding for this work.

AI Declaration

During the preparation of this manuscript, the author used ChatGPT for writing assistance and manuscript refinement, and Perplexity AI for literature exploration and background research. During the implementation stage

of the project, the author used Cursor, ChatGPT Codex, and GitHub Copilot for coding assistance, and Gemini for limited recommendations, including assistance with auditing repository artifacts and checking internal consistency between the manuscript and the underlying code and results. All AI-assisted text, code, and analysis were independently verified by the author against the repository’s artifacts before inclusion, including dedicated schema-validation, replay-artifact, and negative-result audits documented in the repository; AI assistance was not used to introduce any positive result that is not directly supported by these verified artifacts. The author takes full responsibility for the content of the manuscript, the code, and the reported results.

Data Availability

The datasets used in this study are cited in the manuscript and were obtained from their respective public or documented sources. The repository associated with this work contains the code used for data preparation, experiment execution, and analysis: <https://github.com/SoroushVahidi/Augmented-caching>. Access to specific datasets remains subject to the terms, licenses, and availability conditions of the original data providers.

Code Availability

The code used to prepare the data, train the models, run the baseline comparisons, and generate the experimental artifacts is available at: <https://github.com/SoroushVahidi/Augmented-caching>

Declaration of Competing Interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Appendix A. Historical Single-Split End-to-End Evaluation

This appendix reports a historical, single-split end-to-end online replay evaluation of `evict_value_v1` that predates the corrected, leakage-free evaluation in Section 3.4. It is retained here for completeness and because it is the only evaluation in this paper covering four reference policies not included

in the corrected matched comparison (PredMk, T&D, BO/LRU, and REST). It is explicitly historical exploratory analysis, not primary performance evidence: the corrected, leakage-free evaluation in Section 3.4 is the primary evidence for `evict_value_v1`’s comparative performance in this paper.

Appendix A.1. End-to-End Online Replay Evaluation Across Available Capacities

In addition to the offline ablation in Section 3.3, we report end-to-end online replay results for all eight policies in Table 2, evaluated under the common simulator described in Section 3.1 across the seven trace families in Table 1, at 50,000 requests per trace. At the time of writing, this evaluation has been completed at cache capacities 32, 64, and 128 slots; capacity 256 has not yet been evaluated end-to-end and is not reported in this paper. We are explicit about this boundary because the capacity-128 result discussed below is itself non-monotonic relative to capacities 32 and 64, and we judged it more important to report and analyze the existing three-point trend honestly than to extend further computation into a larger capacity sweep before doing so.

Table A.8 reports mean replay misses, averaged across the seven trace families, for each policy at each evaluated capacity, and Fig. A.3 visualizes the same data together with `evict_value_v1`’s relative gap against the three strongest lightweight baselines (LRU, SIEVE, and FIFO-Reinsertion).

We disclose an important protocol limitation of this specific evaluation directly here rather than only in the Limitations section: the `evict_value_v1` model underlying Table A.8 was fit with a chunk-level train/validation/test split applied within each trace’s own full $[0, 50,000)$ request range, rather than on a held-out trace family with zero overlap with the evaluated stream. This means the reported numbers for `evict_value_v1` in this specific table do not rule out train/test overlap and should not be read as a clean estimate of its generalization performance; the seven non-learned or fixed reference policies in this table (LRU, SIEVE, FIFO-Reinsertion, PredMk, T&D, BO/LRU, REST) are unaffected, since they are either parameter-free or do not fit trace-specific models on this data. Section 3.4 reports a corrected, leakage-free evaluation of `evict_value_v1` using leave-one-family-out training, together with the direct comparison against LRB, 3L-Cache, CACHEUS, and HALP requested for this revision; that corrected evaluation is the primary evidence for `evict_value_v1`’s comparative performance in this paper. We retain Table A.8 because it is the only evaluation in this paper covering PredMk, T&D, BO/LRU, and REST, and because, notably,

the potential overlap did not produce a favorable bias here: `evict_value_v1` still underperforms every other policy in this table despite it.

Table A.8: Historical single-split end-to-end online replay evaluation: mean misses across the seven trace families in Table 1 (50,000 requests/trace), at each evaluated cache capacity. Capacity 256 has not yet been evaluated end-to-end. This table predates the corrected leave-one-family-out protocol of Section 3.4 and is retained for its coverage of PredMk, T&D, BO/LRU, and REST.

Policy	Cap. 32	Cap. 64	Cap. 128
LRU	34667.1	33650.1	32495.6
SIEVE	35313.6	34362.7	33279.3
FIFO-Reinsertion	34688.0	33672.1	32530.6
PredMk	34843.9	33773.1	32664.6
T&D	35087.0	33998.7	32928.9
BO/LRU	34668.6	33651.3	32496.9
REST	34667.1	33650.1	32495.6
EV (proposed)	36344.1	34409.0	36316.6

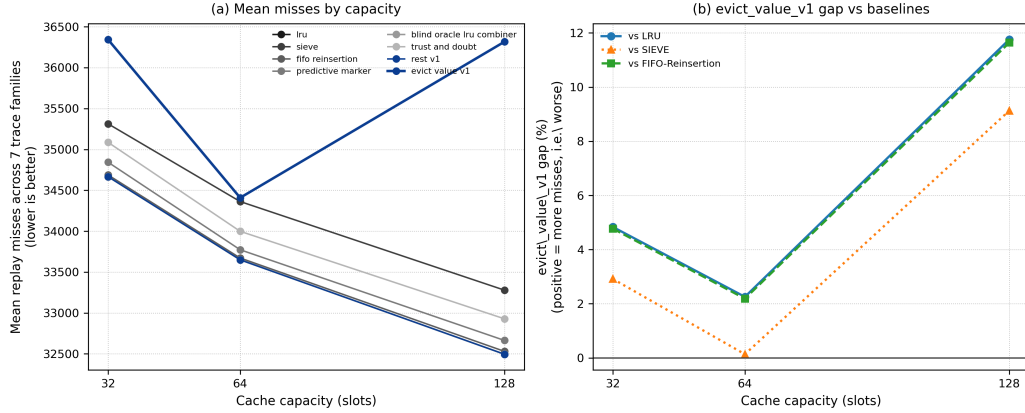


Figure A.3: Capacity-trend comparison across the available evaluated capacities (32, 64, 128; capacity 256 not evaluated in this revision), from the historical single-split evaluation described above. (a) Mean replay misses per policy at each capacity, averaged across the seven trace families. (b) `evict_value_v1`'s relative miss gap against LRU, SIEVE, and FIFO-Reinsertion at each capacity; positive values indicate more misses than the baseline. The LRU and FIFO-Reinsertion gap curves nearly coincide because the two baselines have nearly identical miss counts in the averaged replay; distinct line styles and markers are used to keep them visually separable. The non-monotonic widening at capacity 128 is discussed in Section 3.8 and Section 4.3.

Three observations follow directly from this available-capacity replay.

First, `evict_value_v1` does not improve on LRU, SIEVE, FIFO-Reinsertion, or REST on average at any of the three evaluated capacities: its mean-miss gap relative to LRU is +4.84% at capacity 32, +2.26% at capacity 64, and +11.76% at capacity 128 (equivalently +2.92%, +0.13%, and +9.13% against SIEVE, and +4.77%, +2.19%, and +11.64% against FIFO-Reinsertion). REST is numerically tied with LRU at all three evaluated capacities, so the same average gap applies there as well. This is a negative end-to-end result for the proposed policy relative to the strongest baselines in our pool, and we report it directly rather than restricting the paper’s empirical claims to the offline ablation evidence above.

Second, the other reference policies cluster tightly around LRU at every evaluated capacity. `blind_oracle_lru_combiner` (BO/LRU) and `rest_v1` (REST) are numerically indistinguishable from LRU (0.00% gap) at all three capacities, FIFO-Reinsertion stays within 0.11% of LRU throughout, and SIEVE, T&D, and PredMk remain within roughly 0.4–2.4% of LRU at every capacity. `evict_value_v1` is the only policy in the comparison whose gap exceeds 5% at any evaluated capacity, and the only one whose relative position worsens between capacity 64 and capacity 128 rather than tracking the otherwise stable ordering of the other seven policies.

Third, the capacity trend for `evict_value_v1` is non-monotonic: the gap narrows from capacity 32 to capacity 64 and then widens sharply at capacity 128, rather than improving or degrading smoothly with capacity. We examine this pattern at the workload level next, and return to its likely cause in the Discussion and Limitations.

Appendix A.2. Workload-Specific Breakdown

Table A.9 reports `evict_value_v1`’s relative miss gap against LRU separately for each of the seven trace families at each evaluated capacity, rather than only as the family-averaged numbers in Table A.8.

Two patterns stand out. First, Wikimedia pageviews is degenerate at every evaluated capacity, as noted in the table, and we exclude it from substantive interpretation. CloudPhysics and MetaKV remain close to LRU at every capacity (at most +3.10% and +1.70%, respectively), consistent with the family-averaged picture in Table A.8.

Second, the capacity-128 widening is concentrated in, but not limited to, two families. BrightKite and Citi Bike show the largest gaps at capacity 128 (+50.91% and +45.49%, respectively), up sharply from roughly +10% or less at capacities 32 and 64. MetaCDN and Twemcache also show double-digit

Table A.9: `evict_value_v1` relative miss gap against LRU, by trace family and evaluated capacity, from the historical single-split evaluation of Appendix Appendix A. Positive values indicate more misses than LRU (worse).

Trace family	Cap. 32	Cap. 64	Cap. 128
BrightKite	+10.47%	+10.72%	+50.91%
Citi Bike	+7.90%	+4.70%	+45.49%
CloudPhysics	+0.72%	+1.57%	+3.10%
MetaCDN	+22.37%	+2.73%	+17.62%
MetaKV	+1.70%	+0.01%	+0.76%
Twemcache	+1.84%	+3.16%	+14.06%
Wikimedia [†]	+0.00%	+0.00%	+0.00%

[†]Degenerate at every evaluated capacity: all eight policies reach 50,000/50,000 misses (0% hit rate), because the working set exceeds even the largest evaluated capacity. Not informative for cross-policy comparison.

gaps at capacity 128 (+17.62% and +14.06%), though their capacity-32/64 trajectories differ from BrightKite’s and Citi Bike’s: MetaCDN’s gap is actually largest at capacity 32 (+22.37%), drops to +2.73% at capacity 64, and rises again at capacity 128, a U-shaped pattern rather than a monotonic one, while Twemcache’s gap increases steadily across all three capacities (+1.84%, +3.16%, +14.06%). In total, four of the six non-degenerate families show a capacity-128 gap above 10%, while the remaining two (CloudPhysics, MetaKV) stay below 3.1% throughout. We therefore describe the capacity-128 degradation as broad-based rather than as an artifact specific to two outlier traces, while still noting that BrightKite and Citi Bike are the most extreme cases.

Taken together with the offline ablation in Section 3.3, the capacity-128 degradation documented above is broad-based across workload families rather than an artifact of a single outlier trace: four of the six non-degenerate families show a capacity-128 gap above 10%, reinforcing the negative end-to-end result already reported in this appendix. We discuss its likely cause in Section 3.8 and revisit it in Section 4.3.

Appendix B. Guarded Fallback Mechanism: Full Formalism

This appendix gives the full formal specification of the guarded fallback mechanism summarized in Section 2.3. As stated there, this mechanism is an unvalidated implementation extension: its end-to-end effect has not been

measured in this study, and it is not used to support any quantitative claim in this paper.

The guard uses repeated *very-early-return* evictions as its signal. Let q_t^* denote the victim proposed by the learned scorer at time t . If the evicted item is requested again within a short window, the decision is treated as suspicious because it indicates that an item with immediate reuse value may have been removed.

Formally, let

$$E_t = \begin{cases} 1, & \text{if the item evicted at time } t \text{ returns within } W \text{ subsequent requests,} \\ 0, & \text{otherwise,} \end{cases} \quad (\text{B.1})$$

where W is a small early-return window. The mechanism aggregates these suspicious events over a sliding interval of length T . Writing the recent suspicious count as

$$S_t = \sum_{\tau=\max(1, t-T+1)}^t E_\tau, \quad (\text{B.2})$$

the trigger condition is

$$S_t \geq M, \quad (\text{B.3})$$

where M is the threshold required to enter fallback mode.

Once triggered, the controller temporarily delegates the eviction choice to a conservative fallback policy for a bounded duration D . In the current implementation, the fallback is configurable and may be instantiated by a simple baseline such as least-recently used eviction or a marker-style policy. Let $\mathcal{M}_t \in \{\text{BASE}, \text{FALLBACK}\}$ denote the operating mode at time t . The final victim is then

$$\tilde{q}_t = \begin{cases} q_t^*, & \text{if } \mathcal{M}_t = \text{BASE}, \\ q_t^{\text{fb}}, & \text{if } \mathcal{M}_t = \text{FALLBACK}, \end{cases} \quad (\text{B.4})$$

where q_t^{fb} is the victim selected by the fallback controller. After D requests in fallback mode, the system returns to base mode unless the trigger condition fires again.

This mechanism is intentionally lightweight and heuristic: trigger signal, thresholds, duration, and fallback choice are implementation decisions for controlled study, not a theorem-backed robustness guarantee [6, 7].

Table C.10: Decision-time feature groups used by the eviction-value scorer. The table summarizes the main feature families implemented in the current artifact pipeline; the full feature specification is repository-based and reproducibility-oriented.

Feature group		Role in decision-time scoring
Request-side features	fea-	Encode properties of the current request and its immediate decision context.
Candidate-side reuse and recency features		Summarize candidate age, recent reuse behavior, and local history relevant to eviction risk.
Predictor–LRU comparison	fea-	Capture how predictive context differs from recency-based signals for the same candidate.
Cache-state summary features	sum-	Describe the current cache composition and local competition among candidates.
Disagreement	fea-	Indicate whether predictor-oriented and recency-oriented views point to different eviction choices.
Recent-activity	fea-	Summarize short-window request activity and local workload dynamics near the decision point.

Appendix C. Decision-Time Feature Groups

References

- [1] L. A. Belady, A study of replacement algorithms for virtual-storage computer, *IBM Systems Journal* 5 (2) (1966) 78–101. doi:10.1147/SJ.52.0078.
- [2] A. Fiat, R. M. Karp, M. Luby, L. A. McGeoch, D. D. Sleator, N. E. Young, Competitive paging algorithms, *Journal of Algorithms* 12 (4) (1991) 685–699. doi:10.1016/0196-6774(91)90041-V.
- [3] T. Lykouris, S. Vassilvitskii, Competitive caching with machine learned advice, *Journal of the ACM* 68 (4) (2021) 24:1–24:25. doi:10.1145/3447579.
- [4] A. Antoniadis, C. Coester, M. Eliáš, A. Polak, B. Simon, Online metric algorithms with untrusted predictions, in: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, Vol. 119 of *Proceedings of Machine Learning Research*, PMLR, 2020, pp. 345–355.

- [5] S. Im, R. Kumar, A. Petety, M. Purohit, Parsimonious learning-augmented caching, in: Proceedings of the 39th International Conference on Machine Learning (ICML), Vol. 162 of Proceedings of Machine Learning Research, PMLR, 2022, pp. 9588–9601.
- [6] A. Wei, Better and simpler learning-augmented online caching, in: Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2020), Vol. 176 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 60:1–60:17. doi:10.4230/LIPIcs.APPROX/RANDOM.2020.60.
- [7] J. Chłędowski, A. Polak, B. Szabucki, K. Żoła, Robust learning-augmented caching: An experimental study, in: Proceedings of the 38th International Conference on Machine Learning (ICML), Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 1920–1930.
- [8] M. Mitzenmacher, S. Vassilvitskii, Algorithms with predictions, Communications of the ACM 65 (7) (2022) 33–35. doi:10.1145/3528087.
- [9] D. Rohatgi, Near-optimal bounds for online caching with machine learned advice, in: Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, 2020, pp. 1834–1845. doi:10.1137/1.9781611975994.112.
- [10] N. Bansal, C. Coester, R. Kumar, M. Purohit, E. Vee, Learning-augmented weighted paging, in: Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms (SODA), SIAM, 2022, pp. 67–89.
- [11] A. Blum, C. Burch, On-line learning and the metrical task system problem, Machine Learning 39 (1) (2000) 35–58. doi:10.1023/A:1007621832648.
- [12] E. Z. Liu, M. Hashemi, K. Swersky, P. Ranganathan, J. Ahn, An imitation learning approach for cache replacement, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 6237–6247.
URL <https://proceedings.mlr.press/v119/liu20f.html>

- [13] Z. Song, D. S. Berger, K. Li, W. Lloyd, Learning relaxed belady for content distribution network caching, in: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), USENIX Association, 2020, pp. 529–544.
URL <https://www.usenix.org/conference/nsdi20/presentation/song>
- [14] X. Hu, E. Ramadan, W. Ye, F. Tian, Z.-L. Zhang, Raven: Belady-guided, predictive (deep) learning for in-memory and content caching, in: Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '22), ACM, 2022, pp. 72–90. doi:10.1145/3555050.3569134.
URL <https://doi.org/10.1145/3555050.3569134>
- [15] I. Shah, A. Jain, C. Lin, Effective mimicry of belady’s MIN policy, in: 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), IEEE, 2022, pp. 558–572. doi:10.1109/HPCA53966.2022.00048.
- [16] Z. Song, K. Chen, N. Sarda, D. Altinbüken, E. Brevdo, J. Coleman, X. Ju, P. Jurczyk, R. Schooler, R. Gummadi, Halp: Heuristic aided learned preference eviction policy for youtube content delivery network, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), USENIX Association, 2023, pp. 1149–1163.
URL <https://www.usenix.org/conference/nsdi23/presentation/song-zhenyu>
- [17] W. Zhou, Z. Niu, Y. Xiong, J. Fang, Q. Wang, 3l-cache: Low overhead and precise learning-based eviction policy for caches, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, 2025, pp. 237–254.
URL <https://www.usenix.org/conference/fast25/presentation/zhou-wenbin>
- [18] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, G. Narasimhan, Learning cache replacement with cacheus, in: 19th USENIX Conference on File and Storage Technologies (FAST 21), USENIX Association, 2021, pp. 341–354.
URL <https://www.usenix.org/conference/fast21/presentation/rodriguez>

- [19] A. N. Elmachtoub, P. Grigas, Smart “predict, then optimize”, *Management Science* 68 (1) (2022) 9–26. doi:10.1287/mnsc.2020.3922.
- [20] B. Wilder, B. Dilkina, M. Tambe, Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33, 2019, pp. 1658–1665. doi:10.1609/aaai.v33i01.33011658.
- [21] S. Shah, K. Wang, B. Wilder, A. Perrault, M. Tambe, Decision-focused learning without decision-making: Learning locally optimized decision losses, in: *Advances in Neural Information Processing Systems* 35 (NeurIPS 2022), 2022.
URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/0904c7edde20d7134a77fc7f9cd86ea2-Abstract-Conference.html
- [22] O. E. Balghiti, A. N. Elmachtoub, P. Grigas, A. Tewari, Generalization bounds in the predict-then-optimize framework, *Mathematics of Operations Research* 48 (4) (2023) 2043–2065. doi:10.1287/moor.2022.1330.
URL <https://doi.org/10.1287/moor.2022.1330>
- [23] G. Tolomei, L. Takanen, F. Pinelli, Mustache: Multi-step-ahead predictions for cache eviction, *CoRR* abs/2211.02177 (2022).
URL <https://arxiv.org/abs/2211.02177>
- [24] D. Yang, D. S. Berger, K. Li, W. Lloyd, A learned cache eviction framework with minimal overhead, *arXiv preprint arXiv:2301.11886* (2023). doi:10.48550/arXiv.2301.11886.
URL <https://arxiv.org/abs/2301.11886>
- [25] J. Yang, Z. Mao, Y. Yue, K. V. Rashmi, GL-Cache: Group-level learning for efficient and high-performance caching, in: *21st USENIX Conference on File and Storage Technologies (FAST ’23)*, USENIX Association, 2023, pp. 115–134.
URL <https://www.usenix.org/conference/fast23/presentation/yang-juncheng>
- [26] H. Xia, W. Nixon, B. D. Marthen, P. Bhandari, J. Yang, Learning-augmented heuristics: Simple yet smart, robust and interpretable cache eviction, in: *20th USENIX Symposium on Operating Systems Design*

- and Implementation (OSDI '26), USENIX Association, 2026, pp. 2241–2259.
URL <https://www.usenix.org/conference/osdi26/presentation/xia>
- [27] L. Li, J. Guo, Y. Fan, J. Wu, Z. Wang, J. Zhang, Y. Tamir, X. Wang, Y. Luo, D. Zhou, Merlin: An efficient adaptive cache eviction algorithm via fine-grained characterization, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26), USENIX Association, 2026, pp. 2223–2239.
URL <https://www.usenix.org/conference/osdi26/presentation/li-liujia>
- [28] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, K. V. Rashmi, Fifo queues are all you need for cache eviction, in: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23), ACM, 2023. doi:10.1145/3600006.3613147.
URL <https://doi.org/10.1145/3600006.3613147>
- [29] Y. Zhang, J. Yang, Y. Yue, Y. Vigfusson, K. V. Rashmi, SIEVE is simpler than LRU: An efficient turn-key eviction algorithm for web caches, in: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), USENIX Association, 2024, pp. 1229–1246.
URL <https://www.usenix.org/conference/nsdi24/presentation/zhang-yazhuo>
- [30] E. Cho, S. A. Myers, J. Leskovec, Friendship and mobility: User movement in location-based social networks, in: Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2011, pp. 1082–1090. doi:10.1145/2020408.2020579.
- [31] Citi Bike, Citi bike system data, accessed: 2026-04-10 (2025).
URL <https://citibikenyc.com/system-data>
- [32] J. Yang, Open-source cache dataset, gitHub repository. Accessed: 2026-04-10 (2024).
URL https://github.com/cacheMon/cache_dataset
- [33] Twitter, Anonymized cache request traces from twitter production, official trace repository; accessed 2026-08-13 (2020).
URL <https://github.com/twitter/cache-trace>

- [34] J. Yang, Y. Yue, K. V. Rashmi, A large scale analysis of hundreds of in-memory cache clusters at twitter, in: 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20), USENIX Association, 2020, pp. 191–208.
URL <https://www.usenix.org/conference/osdi20/presentation/yang>
- [35] Wikimedia Foundation, Analytics datasets: Pageviews, accessed: 2026-04-10.
URL <https://dumps.wikimedia.org/other/pageviews/readme.html>